



JAVA using BlueJ



Methods/Function

- A function is a block containing a number of statements, which perform a specific and well defined task(s).
- Every function has a unique name which is used to identify it in a program.
- Once a task is packaged in a method, it can be called from anywhere in a program through other statements.

- Function reduce the **complexity** of program by dividing the into smaller modules, thus making them easier to read and debug.
- Function enhance **reusability** of code and can be called from any where in a program and any number of time.
- Function are useful in hiding the implementation details.
- It is easier to edit a program, which is written using methods.
- While working on one function, one can focus on just that part of the program by debugging, constructing and making it perfect.

- Function can be broadly classified into two categories:
 - **Built-in function or library functions:**
 - These are inbuilt function, which are grouped together and stored in a standard java library.
 - You can make use of these functions to get a predefined output, instead of writing your own code.
 - **User defined Function :**
 - These function are designed by the user to carry out some specific tasks.
 - These name of the method is given by the user according to his/her requirement; hence, it is called a user defined function.

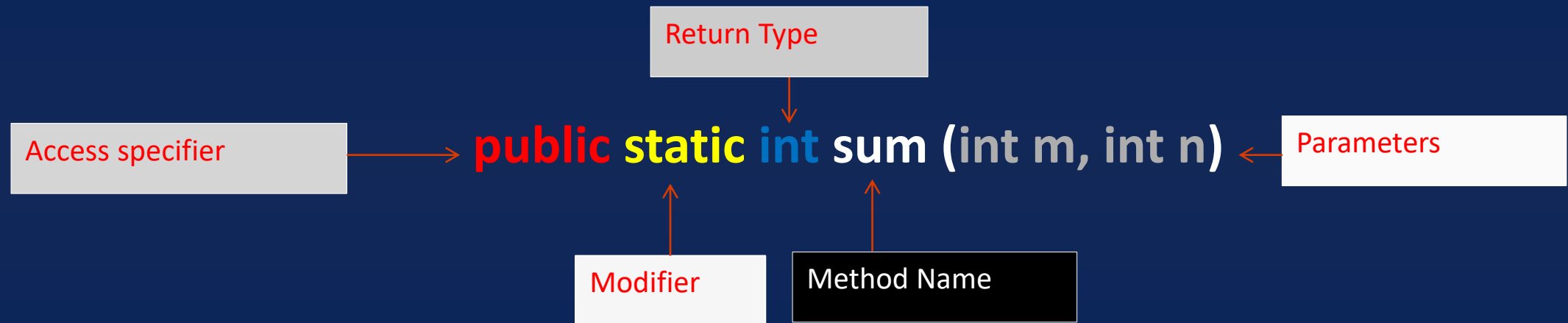
- A method must be defined before it is used anywhere in the program. The general form of method definition consist of two parts.
 - Method Header/prototype
 - Body of method

Method Header { **Access_Specifier** **Modifier** **Return_Type** **Method_Name** (arg1,arg2,arg3)
{
 -- -- -- -- --
 -- -- -- -- --
 -- -- -- -- --
}

} Body of Method

Method Header

- A method header/prototype is the first line of method definition that tells the compiler about the method name, type of value returned by the method, and the types of arguments to be passed, if the function is expected to return some result, then the function prototype reveals the data type of the result.



Categories of Function

- **Method with no argument and no return value**
- **Method with argument and no return value**
- **Method with argument and with return value**
- **Method with no argument and with return value**

Arguments to method/Function

- We can pass two type of arguments to a method
- We can pass any primitive data type argument to a java method, like Boolean, char, byte, char, int, long etc. This method of passing arguments is known as passing arguments by value.
- We can also pass reference data types, like object or array.
- A method can be called in two different ways:
 - **Call by Value or Pass by Value**
 - **Call by Reference or Pass by Reference**

➤ Call By Value :

- When you pass parameters to a method by value, the method copies the value of actual parameter into the formal parameter.
- It creates its own copy of argument values and then uses in the method.

➤ Call by Reference :

- In this technique, actual arguments are passed by reference rather than passing by values.
- When function call is made, the reference of actual parameters is copied into formal parameters.
- If any changes are made in the formal parameters, the original value automatically gets modified. This happens because the program has only one copy of the value.

➤ **PURE(ACCESSOR/GETTER) Methods**

- Pure methods do not change the original state of an object.
- They simply access that values that are passed to them as arguments.
- The function generally return a value to its caller method.
- The return value is either a primitive or a new object created inside the method.
- In simple, we can say that a method does not change the original state of an object an object or simply return the initial state of the object is called pure method.

IMPURE(MUTATOR/PUTTER) Methods

- Impure method change the initial state of an object, every time the are called.
- It means that the values of actual parameters get modified or change or changed depending upon the current state of the object on which the method operates.

- Function overloading, also known as method overloading, is a feature in Java that allows a class to have multiple methods with the same name but different parameter lists.
- When an overloaded method is invoked (called), the java compiler compares the numbers of parameters and the data types of the arguments with the parameters of the different versions of the method. The version which matches in both respects. i.e. the numbers and types of the parameters, get invoked.
-

Qus 1: Create a program to demonstrate function Overloading. The program must have three different versions of the function, calculateArea(). This function will calculate the area of circle, rectangle and square on the basis of type and number of arguments passed.

Qus 2: Design a class to overload a function num_cal() as follows:

- void num_cal(int num, char ch) with one integer arguments and one character arguments. It computes the square of an integer if choice ch is 's' otherwise, computes the cube.
- void num_cal(int a, int b, char ch) with tow integer arguments and one character argument. It compute the product of integer arguments if ch is 'p' else adds the integer.
- void num_cal(String str1,String str2) with tow String arguments and prints the two String are equal or not.

- Static methods are also called class methods because these methods can be called without creating an object of that class.
- A method is made static by using the keyword, 'static' in the method declaration.
- We can declare both functions and variables as static.
- The most common example of a static function is the main().
- As you have seen that generally, the main() method of a Java program is declared with the keyword, 'static', so that main() can be executed by the interpreter without creating an object from the class.

- There are three kinds of variables in Java –
 - Local variables
 - Instance variables
 - Class/Static variables

- Local variables are declared in methods, constructors, or blocks.
- Local variables are created when the method, constructor or block is entered and the variable will be destroyed once it exits the method, constructor, or block.
- Access modifiers cannot be used for local variables.
- Local variables are visible only within the declared method, constructor, or block.
- Local variables are implemented at stack level internally.
- There is no default value for local variables, so local variables should be declared and an initial value should be assigned before the first use.


```
public class Test
{
    public void addAMT()
    {
        int amt;
        amt=amt+500;
        System.out.println("Amount is: " + amt);
    }
    public static void main(String args[])
    {
        Test test = new Test();
        test.addAMT();
        test.addAMT();
    }
}
```

- Instance variables are declared in a class, but outside a method, constructor or any block.
- Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.
- Instance variables hold values that must be referenced by more than one method, constructor or block, or essential parts of an object's state that must be present throughout the class.
- Access modifiers can be given for instance variables.
- Instance variables can be accessed directly by calling the variable name inside the class.
- Instance variables have default values. For numbers, the default value is 0, for Booleans it is false, and for object references it is null. Values can be assigned during the declaration or within the constructor.

```
public class Test
{
    int amt=0;
    public void addAMT()
    {
        amt=amt+500;
        System.out.println("Amount is: " + amt);
    }
    public static void main(String args[])
    {
        Test test = new Test();
        test.addAMT();
        test.addAMT();
    }
}
```

- Class variables also known as static variables are declared with the static keyword in a class, but outside a method, constructor or a block.
- There would only be one copy of each class variable per class, regardless of how many objects are created from it.
- Static variables are created when the class is loaded in memory.
- Static variables can be accessed by calling with the class name *ClassName.VariableName*.
- Example:

Example: Class/Static Variables

```
class Counter
{
    static int count=0;
    void addCount()
    {
        count++;
        System.out.println(count);
    }
    public static void main(String args[])
    {
        Counter c1=new Counter();
        c1.addCount();
        Counter c2=new Counter();
        c2.addCount();
    }
}
```

- 1. Once a function is defined, it can be invoked repeatedly. What is this feature called?
 - a. Interface
 - b. Reusability
 - c. Restructuring
 - d. All of these
- 2. If a function does not return any value, its return type should be:
 - a. int
 - b. no-return
 - c. void
 - d. empty
- 3. A function that computes a value and returned is called:
 - a. Computational function
 - b. Manipulative function
 - c. Procedural function
 - d. None of these
- 4. A type of parameter that are used to identify what data is to be passed to a function is called:
 - a. Formal parameter
 - b. Actual parameter
 - c. Both a and b
 - d. None of these
- The parameter list in a function definition is called:
 - a. Function prototype
 - b. Function signature
 - c. Both a and b
 - d. None of these

- The first line of the function definition is called:
 - a. Function prototype
 - b. Function signature
 - c. Both a and b
 - d. None of these
- The number of values that a function can return is:
 - a. 1
 - b. 2
 - c. 3
 - d. 4
- A local variable in a function has its scope:
 - a. Limited within the function
 - b. Can be accessed anywhere within the same class
 - c. No limitation at all
 - d. None of these
- Which among the following is a valid name for a function?
 - a. function
 - b. 2function
 - c. fun in action
 - d. fun#
- Which among the following is not a valid access specifier?
 - a. public
 - b. private
 - c. partially
 - d. protected